

Bases de données documentaires et distribuées
Cours NFE04
Partitionnement par hachage

Auteurs : Raphaël Fournier-S'niehotta, Philippe Rigaux, Nicolas Travers
prénom.nom@cnam.fr

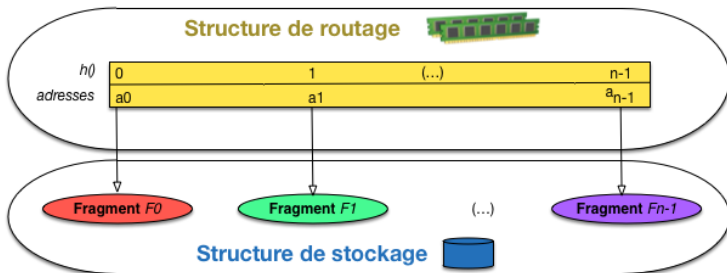
Département d'informatique
Conservatoire National des Arts & Métiers, Paris, France

Plan du cours

- 1 Principes
- 2 Le hachage cohérent

Principe du partitionnement par hachage

On prend la clé ; on l'interprète comme un entier, on calcule le reste r de la division par N : le document est affecté au fragment r .



Remarque

Fontionne très bien pour N , le nombre de fragments, fixe.

Les opérations

Implantation très simple et très efficace.

- $get(i)$: calculer $r = h(i)$, et accéder au fragment dont l'adresse est a_r , chercher le document en mémoire ;
- $put(i, d)$: calculer $r = h(i)$, insérer d dans le fragment dont l'adresse est a_r ;
- $delete(i)$: comme la recherche, avec effacement du document trouvé ;
- $range(i, j)$: **pas possible avec une structure par hachage**, il faut faire un parcours séquentiel complet.

Remarque

Si N change : tout devient faux.

Le problème

Tout repose sur l'hypothèse que la fonction $h()$ est **immuable**.

Soit $N = 5$, et un flux continu d'insertion et de recherche de documents, parmi lesquelles l'insertion, puis la recherche de l'identifiant 17.

- on effectue `*put(17, d)*`, d est inséré dans F_2 (tout le monde suit ?) ;
- les insertions continuent, jusqu'à la nécessité d'ajouter un sixième fragment : maintenant $h(i) = i \bmod 6$.
- on effectue `get(17)`, on cherche dans F_5 (vous voyez pourquoi ?) qui ne contient pas d .

Pas de solution simple !!

Systèmes distribués basés sur le hachage

Idée simple : tout le monde utilise la même fonction de hachage, les “blocs” sont les serveurs, et tout marche bien.

Presque ! A ceci près que :

- le système est nécessairement **dynamique** ; il faudra ajouter ou retirer des serveurs.
- attention aux clients qui pourraient utiliser une fonction de hachage **obsolète** : il faut leur garantir que leurs requêtes aboutissent quand même.

En 1997, un article a proposé une solution maintenant très largement utilisé : le **hachage cohérent** ou *Consistent hashing*.

- D'abord conçu et utilisé pour des **caches distribués** (*memcached*)
- Popularisé pour la gestion de données par le système Dynamo (Amazon, 2007)
- Maintenant intégré à de très nombreux systèmes : Voldemort, Riak, Chord, ...

Plan du cours

- 1 Principes
- 2 Le hachage cohérent

Hachage cohérent (*consistent hashing*)

Soit N le nombre de serveurs. La fonction suivante

$$\text{hash}(\text{key}) \rightarrow \text{modulo}(\text{key}, N) = i$$

associe une paire $(\text{key}, \text{value})$ au S_i .

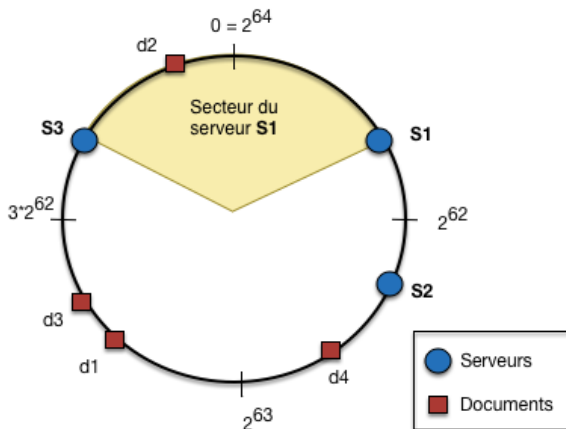
Problème : si N change, ou si un client utilise une valeur incorrecte de N , le hachage devient **incohérent**.

Avec le **Consistent hashing**, ajouter ou retirer un serveur n'affecte la collection que de façon **marginale**.

- on prend une fonction **permanente**, h , appliquée aux clés **et** aux serveurs vers l'espace d'adressage $A = [0, 2^{64} - 1]$;
- A est organisé comme un anneau parcouru dans le sens des aiguilles d'une montre ;
- si S et S' sont deux serveurs adjacents sur l'anneau, toutes les clés de l'intervalle $]h(S), h(S')]$ appartiennent à S' .

Illustration

Serveurs **et** documents sont placés sur l'anneau.

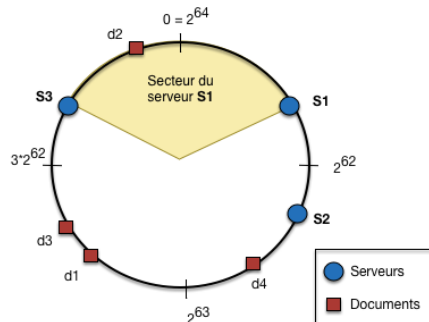


Notez le déséquilibre des fragments (pour l'instant).

La table de hachage

Elle établit une correspondance entre le découpage de l'anneau en arcs de cercle, et l'association de chaque arc à un serveur.

Intervalle	Serveur
$]c, a]$	S1
$]a, b]$	S2
$]b, c]$	S3



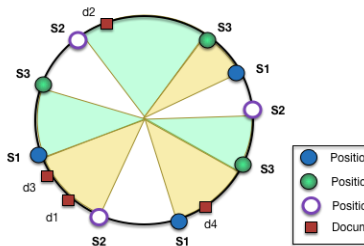
Quelques détails vraiment utiles

Comment fonctionne la tolérance aux pannes ? Comment fonctionne l'équilibrage ?

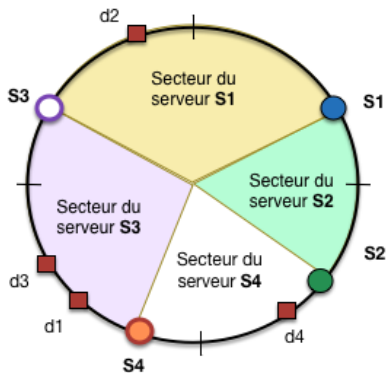
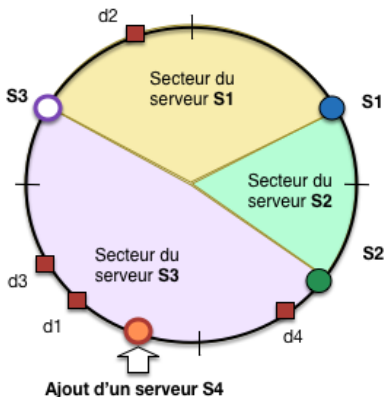
Panne $\Rightarrow$ géré par la réplication (surprise!) ;
par exemple on copie sur la machine suivante
dans l'anneau, etc.

Équilibrage $\Rightarrow$ un même serveur (physique)
est distribué en plusieurs points (virtuels) sur
l'anneau.

- plus il y a de points, plus le serveur recevra de données ;
- en cas de panne, réorganisation répartie plus justement.



Ajout et suppression de serveurs



Une réorganisation **locale** est suffisante (S_3 transmet à S_4).

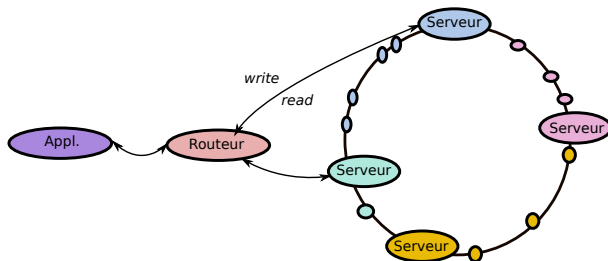
Plan du cours

- 1 Principes
- 2 Le hachage cohérent

Mais où est donc la table de hachage ?

Qui nous dit quelle est la liste des serveurs et leur intervalle ?

Il nous faut un **routeur**.



Autres solutions : (i) la table est répliquée sur chaque nœud (acceptable quand on n'ajoute/retire pas de serveurs tout le temps), (ii) le routeur est intégré au client.